LAB.4

1.

Stos (Stack): Pamięć automatyczna i szybka (dla zmiennych lokalnych). Sama się czyści.
Sterta (Heap): Pamięć dynamiczna, zarządzana ręcznie (**malloc/free**). Jest ogromna, ale wymaga pilnowania wycieków.

2.

malloc: Rezerwuje pamięć, ale zostawia w niej **"śmieci"**(stare dane).
calloc: Rezerwuje pamięć i od razu wypełnia ją zerami.

3.

Wypizse **10.**
Prawidłowo alokuje pamięć, wpisuje tam 10, wypisuje na ekran i poprawnie zwalnia **free**.

4-5(połączone)

DynamicAllocation.cpp

C: > Users > hamul > Desktop > Completed CDV modules > Programowanie nieskopoziomowe(C

```cpp
#include <iostream>
#include <cstdlib>

using namespace std;

int main()
{
    int n = 5;

    int *tab = (int*)malloc(n * sizeof(int));
    if (tab == NULL) return 1;

    int suma = 0;
    for(int i = 0; i < n; i++)
    {
        tab[i] = (i + 1) * 2;
        suma += tab[i];
    }

    float srednia = (float)suma / n;
    cout << "Srednia wynosi: " << srednia << endl;

    int nowyRozmiar = 10;
    int *temp = (int*)realloc(tab, nowyRozmiar * sizeof(int));

    if (temp != NULL)
    {
    tab = temp; // Aktualizujemy wskaźnik, jeśli się udało
    }
    free(tab);

    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Srednia wynosi: 6
```

Powyższy program demonstruje pełny cykl zarządzania pamięcią dynamiczną. Najpierw za pomocą **malloc** alokowana jest tablica, na której wykonywane są obliczenia (wypełnienie danymi i wyliczenie średniej). Następnie jej rozmiar jest bezpiecznie powiększany za pomocą **realloc**.

Co, gdy realloc zwróci NULL (Zadanie 5)?
Oznacza to brak wolnej pamięci w systemie. W takiej sytuacji stary blok pamięci pozostaje nienaruszony. Dlatego w kodzie wynik **realloc** przypisano do tymczasowego wskaźnika **(temp)**. Gdybyśmy przypisali wynik bezpośrednio do **tab** i otrzymali **null**, bezpowrotnie stracilibyśmy adres do starej pamięci, co doprowadziłoby do wycieku pamięci. Na koniec programu pamięć jest prawidłowo zwalniana instrukcją **free**.

6.

Błąd: Wyciek pamięci.
Zniszczyliśmy wskaźnik przed zwolnieniem pamięci. **free(nullptr)** nic nie robi, a zaalokowane dane zostają zablokowane w RAM-ie na zawsze.

7.

Błąd: Podwójne zwolnienie**.**
Próba oddania tej samej pamięci dwa razy. Zazwyczaj powoduje natychmiastowy crash programu.

8.

Tak bezwzględnie.
Jeśli brakuje pamięci, **malloc** zwraca **null.** Zapisanie danych pod **null** zawiesie program. Zawsze używamy **if (p == null).**

9.

Błąd: Zwracanie wiszącego wskaźnika.
Funkcja zwalnia pamięć **free**, a potem zwraca do niej adres. W **main** ten adres będzie wskazywał na nieaktywne śmieci.

10.

Memory leak: Zaalokowano pamięć, ale zapomniano wpisać **free().**
Danling pointer: Wskaźnik, który patrzy na adres już zwolniony przez **free().**
Double free: Użycie **free()** dwa razy na tym samym adresie.
Use-after-free: Próba odczytu/zapisu danych w pamięci, która została już zwolniona.

Concepts.cpp

C: > Users > hamul > Desktop > Concepts.cpp > main()

```cpp
#include <stdlib.h>
#include <stdio.h>

int main() {
    // 1.Memory leak
    int *wyciek = (int*)malloc(sizeof(int));
    wyciek = NULL; // Tracimy adres

    // ----------------------------------------------------

    int *p = (int*)malloc(sizeof(int));
    *p = 100;

    free(p); // Zwalniamy pamięć prawidłowo.
    // 2. DANGLING POINTER .

    // 3. USE-AFTER-FREE
    *p = 50;
    // 4. DOUBLE FREE
    free(p);

    return 0;
}
```